Lintel: Product Requirements v2

Oct 21, 2026 · @Richard

Lintel is the supervised job-application agent from the ApplyRelay PRD, rebuilt around one idea: the agent carries the repetitive load, and every word that leaves your hands is grounded in a fact you confirmed. v2 keeps v1's safety spine, fixes the technical gaps found in the audit, moves execution to the user's own browser, and adds the features that decide whether people actually get interviews.

What v2 changes

The audit found eight problems in v1, and each has a fix below. Anything not listed here carries over from v1 unchanged.

#	Problem in v1	Why it matters	v2 fix
1	The last step assumes a hosted agent can click Submit. Greenhouse's application endpoint needs the employer's own API key, and testers report CAPTCHA walls on some Greenhouse and Workday tenants.	The final step fails on the exact sites the demo needs.	Default mode becomes assisted fill with a Submit Guard. The user's own click is the final action. Agent-clicked submit stays only on reviewed destinations with no bot challenge.
2	Hosted remote-browser takeover, stored login state and datacenter browsers (v1 sections 18.5, 21.4, 25.1).	The heaviest engineering and liability surface, for the least user value.	A Local Runner opens a headed browser on the user's machine. MFA and CAPTCHA are handled by the person, in their own window. Hosted runner moves to a later release.

#	Problem in v1	Why it matters	v2 fix
3	Approval hashes a review snapshot, with a vague allowance for nonmaterial DOM change.	Hashes over page structure break on every site tweak.	Approval binds to a canonical payload: answers, documents, consents, destination. DOM drift is handled by re-checking that field values still equal the payload.
4	Exactly-once submit is described, not specified.	A crash after the click is the case that creates duplicates.	Write-ahead submit intent, fencing token, uncertain-by-default recovery.
5	Application and execution are one entity, and the state list is a linear chain with loops.	Retries, resumes and takeovers cannot be audited.	Two layers: the application lifecycle the user sees, and runs, one per execution attempt, each with a step log.
6	Success is measured by preparation and submission mechanics. Nothing targets interviews.	Auto-apply tools see callback rates of roughly 1 to 6 percent, and users report invented skills.	New: fact interview, grounded resume variants, fit briefs, application budget, follow-ups, outcome loop.
7	Resume variants are deferred while WhatsApp and hosted infrastructure come early.	Wrong order of value and risk.	Grounded resume variants move into the MVP. WhatsApp moves to a later release. Jinja gives way to Next.js, and no Redis until measured need.
8	Prompt-injection defense is a principle, not a mechanism.	Page text can still steer a model that reads it.	The planner sees a typed form model built from the accessibility tree, never raw page prose.

Carried forward from v1 unchanged: problem statement (2), non-goals (6), résumé ingestion (10), deduplication (11.3), answer categories and the unknown-answer policy (13), outcome states (17), WhatsApp message rules (19.3 to 19.6, used later), retry rules (27.2). Profile and provenance (9) and retention (26) carry over with the extensions below.

Thesis and name

Lintel sells the opposite of volume. Fully automated tools report callback rates of roughly 1 to 6 percent and expose users to account flags, while copilot tools that keep a human in the loop report 5 to 15 percent. Lintel plays the second game and plays it harder: fewer applications, each one grounded in facts the user confirmed, each one approved by the user before it leaves.

The promise: the agent prepares, you sign off.

The one metric Lintel cannot miss: zero claims in a sent application that the user did not confirm as true.

Why Lintel. A lintel is the beam over a doorway that carries the weight so people can walk through. That is the job here. It takes the load of the repetitive work and the person walks through the door. It is one word, it types cleanly as a CLI (`lintel prepare`), and my searches found no job-search product with the name. That is not a clearance. Before you commit, check domains, app stores, and trademark classes 9, 35 and 42 in India and the US.

Name	Idea	What my check found
Lintel (pick)	The beam that carries the load so you can walk through	No job-search product found
Fairhand	From "fair copy", the clean final version you send	No conflicting product found, weaker story
Countersign	Your second signature approves the application	Taken by an e-signature product
Foothold	A first grip on a new role	Taken by interview-prep apps

Where it sits in the market. Autofill tools prefill and stop. Copilots draft and ask. Fully automated bots submit unattended. Lintel is a copilot with four things the others do not lead with: a fact ledger that every sentence traces to, approval bound to the exact

payload, a guard that checks the form against that approval at the moment of the click, and an outcome loop that learns which applications worked.

Who it serves and the launch wedge

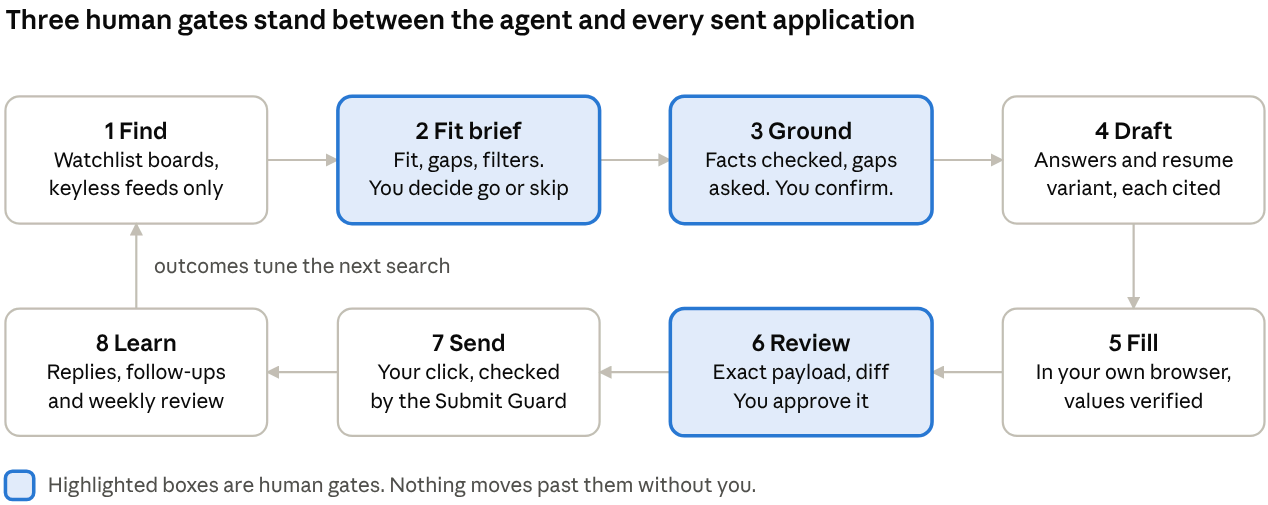
Launch with early-career engineers applying to startups and mid-size tech companies whose career pages run on Greenhouse, Lever or Ashby. The market is crowded, so the wedge has to be narrow and true.

- **Clean discovery.** Greenhouse's job-board GET endpoints need no authentication, Lever serves postings from a public endpoint, and Ashby exposes an unauthenticated job-board endpoint. Discovery needs no scraping and no login.
- **The right tolerance.** Engineers accept a local app and will read an architecture note.
- **Questions where a wrong guess hurts.** Cross-border applicants face work authorization, sponsorship, relocation and notice period on nearly every form. That is where the unknown-answer policy earns its keep.
- **A real first user.** You are user zero. Your own remote search from India for US and UK AI engineering roles is the first live test, and the first source of outcome data.

Out at launch, and why. LinkedIn prohibits bots in its User Agreement, and Workday hides form fields in a way that breaks most autofill, according to Jobscan's review of platform defenses. Indeed adds CAPTCHA walls. These destinations stay draft-only or manual handoff. Recruiters, agencies and high-volume commodity applying remain out of scope, as in v1.

The user-level target (proposed, to be measured). After the first application, a reviewed and truthful packet for a new supported role takes under 10 minutes of the user's time.

The Lintel loop



Lintel loop · 8 steps, 3 human gates

The agent works left to right and stops at the highlighted boxes. Gate 1 is your go or skip on a fit brief. Gate 2 is your confirmation of every fact a draft depends on. Gate 3 is your approval of the exact payload, followed by your own click on the destination form. Outcomes flow back into the next search, so the loop gets better for you specifically.

Features that decide whether people get interviews

v1 perfects the paperwork. These six features go after what the paperwork is for. Jobscan's analysis names tailoring each application to the role as the biggest single factor in outcomes, and tailoring is where competitors drift into invented skills. Lintel tailors from confirmed facts only.

Feature	What it does	Why it moves outcomes	Release
Fact Interview	When a role asks for evidence the ledger lacks, Lintel asks one pointed question. The answer becomes a confirmed fact with a source line.	Many weak applications are weak because true, relevant experience never reached the page.	A

Feature	What it does	Why it moves outcomes	Release
Grounded resume variants	Per role, Lintel proposes a reorder and rewording of existing bullets, shown as a diff against the base résumé. It never adds a claim. The approved variant is stored with its document hash.	Tailoring helps. Fabrication destroys trust, and users report it elsewhere. Pulled forward from v1′s later releases.	A
Fit brief	One screen per role: requirements met with evidence, gaps, uncertain items, conflicts with hard filters, and a recommended action of apply, skip or ask.	Effort goes to roles that can convert.	A
Application budget	The user sets a weekly number of applications. The queue ranks by fit and the surplus waits.	Protects the metric that matters, interviews per application, from the pull toward volume.	A
Outcome loop	Stages from prepared to offer, plus no-reply after a set number of days. Entered by the user at first, detected from forwarded email later. A weekly review shows which variant, source and company size produced replies.	Replaces guesses with the user′s own data.	A manual, later automatic
Follow-up and referral nudges	After silence, Lintel drafts a short follow-up for the user to send. It prompts the user to list contacts at the company and suggests who to ask. Lintel never messages anyone.	Follow-ups and referrals are the human levers that autofill does not touch.	A, then later

A Fact Interview in practice. The résumé mentions a FastAPI service. The role asks for production API experience. Lintel asks: "Did real users call this service? Roughly how many, and over what period?" The user's answer is saved as a reusable fact, tagged with its source, and the draft cites it. If the user answers "no, it was a personal project", the draft says so plainly.

What Lintel does not claim. No feature here promises an interview. The outcome loop exists so the user can see what is working for them instead of trusting a headline number.

Support modes and the destination registry

The registry still decides the mode, never a model. v2 changes what the modes mean, because the final click is where real sites push back.

Mode	Who performs the final click	Runs where	Allowed when
Draft-only	Nobody. Lintel produces a copyable packet.	Lintel only	Default for any unreviewed or prohibited destination
Assisted fill (the default for supported sites)	The user. The Submit Guard checks the form against the approved payload first.	Local Runner, headed browser	The destination is reviewed and enabled in the registry
Approved submission	Lintel, once, after exact approval	Local Runner	Reviewed destination with no bot challenge, enabled per destination. Off at launch except on the Lintel Lab site.

What the three launch platforms actually allow.

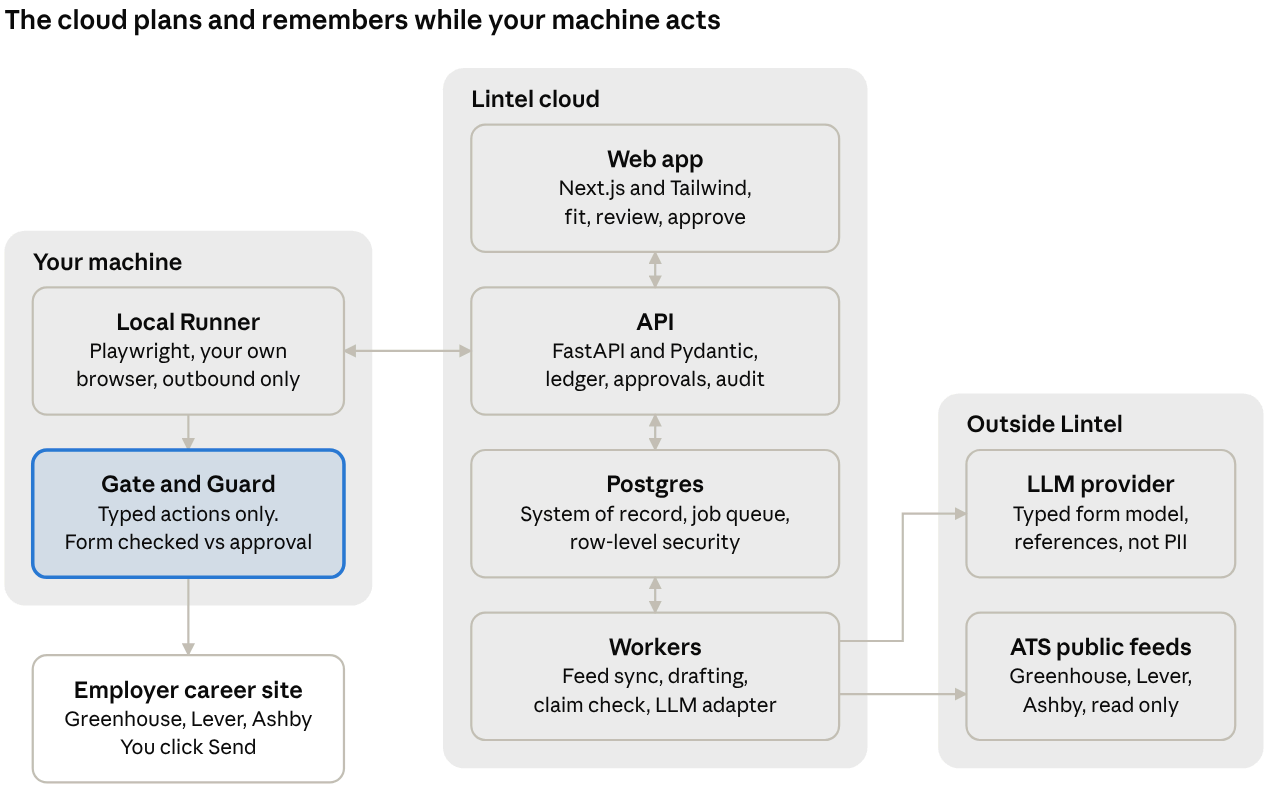
Platform	Discovery	Applying through an API	What Lintel does
Greenhouse	Job-board GET endpoints, no authentication	The submit endpoint needs the employer's Job Board API key	Reads the job and form fields, fills the employer's own form in the user's browser. Testers report CAPTCHA walls on some tenants, and Greenhouse's fraud checks flag data-center IPs, so running from the user's own connection matters.
Lever	Public postings feed at api.lever.co/v0/postings/{site}?mode=json, no key. EU boards live on a separate regional host.	Apply POST needs a key issued by the employer, and the Postings API does not cover custom questions	Reads the posting from the feed, reads the questions from the hosted form, fills in the browser. Checks the response content type, because the feed can return HTML intermittently.
Ashby	Public job-board endpoint per organization, no key, compensation included on request	Authenticated API is reachable only with a key from that employer	Same pattern. Published pay bands feed the fit brief.

Two consequences. First, Lintel is an assisted-fill product by design, and it says so. Second, discovery works from a **company watchlist**. Ashby has no endpoint that lists its customers, so the user adds companies by pasting a careers URL and Lintel detects the

platform and board name. Feeds are polled gently, a few times a day per board, with caching.

Registry record (extends v1). `domain`, `permitted_paths`, `platform`, `adapter_version`, `support_mode`, `bot_challenge_seen`, `policy_review_status`, `policy_review_date`, `known_limitations`, `enabled`, `disabled_reason`. A destination that shows a bot challenge drops to assisted fill or draft-only automatically, and Lintel never tries to solve or evade it.

Architecture: the cloud plans, your machine acts



architecture · local runner, cloud control plane, outside parties

The cloud holds the ledger, the drafts, the approvals and the audit trail. The Local Runner is a small app the user installs and pairs with a device code. It makes outbound calls only, opens a visible browser with the user's own profile, and hands the cloud nothing but step records and redacted evidence. Login state and cookies never leave the machine, which removes v1's heaviest secret-custody and isolation work. The Gate and Guard box is the accent because it is where the product's promise is enforced.

Layer	Choice	Why
Web	Next.js, TypeScript, Tailwind	Review and diff screens need real interactivity. v1's Jinja templates are dropped.
API	FastAPI with Pydantic	One set of schemas shared by the API, the runner and the evals
Data	PostgreSQL with row-level security	Multi-tenancy from day one. Migrations are tested.
Queue	Postgres rows claimed with `FOR UPDATE SKIP LOCKED`	Durable and transactional with state changes. No Redis until measured need.
Agent loop	LangGraph for bounded planning inside one run	Not the source of truth for state, approvals or locks. Postgres is.
Browser	Playwright for Python, headed, in the Local Runner	CAPTCHA and MFA stay with the person. Traces for debugging are redacted.
Models	Provider interface, structured outputs, two roles: extractor and drafter	The model sees field labels and fact summaries, never raw contact details.
Documents	Private encrypted object storage, hash recorded per file	Approvals bind to hashes, not filenames.
Notifications	Email and web push first	WhatsApp arrives in a later release.

Deliberately not built: hosted browser fleets, a vector database, Redis, microservices, Kubernetes, LinkedIn automation, and automatic résumé rewriting without a diff the user approves. Every one of these can be added when a measured problem asks for it.

The trust engine

This is the part that makes Lintel different from every autofill tool, and the part to build most carefully. It has five pieces.

1. Fact ledger

Facts are append-only and immutable. Editing a fact creates a new one that supersedes the old. A profile version is a set of fact ids, so every application can name exactly what it was built from. v1's fields stay, with two additions: `claim_scope` (employment, project, coursework, education, certification) so a personal project can never be written up as a job, and `source_type: user_interview` for answers from the Fact Interview.

2. Claim check on every generated sentence

The drafter must attach `supporting_fact_ids` to each sentence. Then three checks run, in this order, and each fails closed.

1. **Token check (deterministic).** Every number, date, employer, tool name and quantity in a sentence must appear in a cited fact after normalization. This catches the most common fabrication class without trusting a model.
2. **Scope check (deterministic).** A sentence that claims employment must cite employment facts. "Familiar with" never becomes "expert in".
3. **Verifier pass (model).** A separate prompt sees only the sentence and its cited facts and returns supported, overreach or unsupported.

A sentence that fails goes back to the drafter once. If it fails again it becomes a question for the user. The check proves that text traces to confirmed facts. It cannot prove the facts are true, which is why gate 2 belongs to the user.

3. Approval bound to a canonical payload

The approval covers a payload, not a page. Keys are sorted, the encoding is UTF-8, and the hash is SHA-256.

```
{
  "application_id": "app_91",
  "destination": { "platform": "greenhouse", "job_identity": "gh:acme:4412",
"host": "job-boards.greenhouse.io" },
  "profile_version_id": "pv_14",
  "answers": [ { "field_key": "q_backend", "value": "...", "fact_ids":
["f_12", "f_18"] } ],
  "documents": [ { "role": "resume", "sha256": "..." } ],
  "consents": [ { "field_key": "privacy_notice", "value": true } ],
  "action": "submit"
}
```

The approval record holds the payload hash, the runner device id, `approved_at`, `expires_at`, `consumed_at` and `revoked_at`. The proposed lifetime is 30 minutes. Any change to any field produces a new hash and voids the approval. Page structure is deliberately not in the hash. Structural drift is handled by comparing live field values to the payload.

4. Submit Guard (new)

In assisted fill the user clicks the site's own Submit button. The Guard runs in the Local Runner and stands in front of that click.

1. It listens, in the capture phase, for the form's submit event, the final button click and the Enter key.
2. Before the event proceeds it reads every field value, checkbox state and uploaded file hash, normalizes them, and compares them to the approved payload. It also checks that the approval is active and unconsumed and that the job is still open.
3. Match: the event goes through and the submit intent is recorded.
4. Mismatch: the event is blocked and a banner lists the differing fields, with a choice to update the payload and re-approve.

The Guard is a seatbelt, not a lock. The user can always submit on their own, in which case Lintel records `user_reported_submitted` and asks for the confirmation page. Saying this plainly is part of the product's honesty.

5. Exactly-once as far as it can be known

Exactly-once cannot be guaranteed on a third-party site. Lintel guarantees something narrower and testable: it never sends twice on its own authority.

1. Compare-and-set the application from `awaiting_click` to `submitting`, which returns a fencing token.
2. Write a `submit_intent` row carrying the token and the payload hash. This write happens before the click.
3. Let the click through.
4. Observe the outcome through the adapter: a confirmation page, a reference number or a success message.
5. Write the outcome event keyed by application id and token, so a repeat write is a no-op.

A crash after step 2 leaves `submitting` with no outcome. Recovery marks it `submission_uncertain` and never retries. A stale runner holding an old token cannot write an outcome. Outcome states from v1 section 17 carry over, plus `user_reported_submitted`.

Agent harness

v1's principle stands: a deterministic workflow with bounded model help, where the model proposes and the harness decides. v2 turns the principle into mechanisms.

1. **Deterministic form model.** The runner builds a typed `FormModel` from the accessibility tree and DOM: labels, roles, types, options, required flags, character limits, visibility. No model is involved. Each field gets a stable `field_key`, a hash of its section, label and type.
2. **Mapping by handle.** The extractor model receives the `FormModel` and a catalog of fact handles such as `contact.email` or `work[0].employer`. It returns, per field, a handle, `draft_needed`, `ask_user` or `skip`. It never sees the values. The runner resolves handles locally from the approved payload.
3. **Free-text answers** go to the drafter in the cloud with cited facts, then through the claim check.
4. **Policy gate on every action.** An action runs only if the tool is allowed in the current mode, the field exists in the current form model and page version, the value comes from the approved payload, and the target is not a final control. In assisted fill the agent has no submit tool at all.
5. **Verify after every action.** Read the value back, detect validation errors, navigation, unexpected dialogs and expiry. Stop after repeated no-progress steps, as in v1.
6. **Step log without secrets.** Each step records the field key, a hash and length of the value, and the verification result. Playwright traces include DOM snapshots and network data, so they stay on the user's machine unless the user chooses to share one.

What each model sees. This is the data-minimization rule from v1, made specific.

Role	Sees	Never sees
Field mapper	Field labels, types, options, fact-handle names	Raw contact details, page prose

Role	Sees	Never sees
Drafter	The job description and the cited facts for one question	Contact details, other applications
Verifier	One sentence and its cited facts	The job description
Policy gate	Everything, locally, in code	Not a model

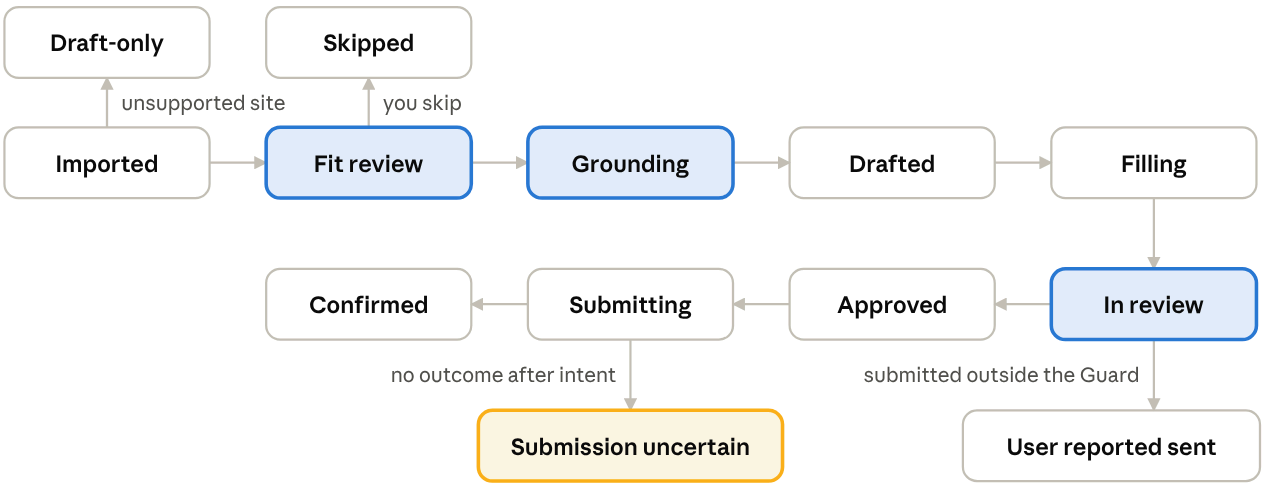
Prompt-injection defense. Page text is data. Three layers hold the line: the planner reads fields, not prose; job-description text reaches only the fit and draft stages, whose output is token-checked and user-reviewed and which cannot change tool availability, because that comes from the mode and the registry; and the runner may navigate only to registry hosts and uploads only documents on the approved list by hash.

Budgets (proposed, carried from v1). At most 50 tool actions per run, two recovery attempts per failed step, 15 minutes of active automation before a pause, one active run per user, and per-user model spend caps.

Lifecycle and run state machines

v1 had one linear state list that mixed what the user sees with what a worker does. v2 splits them. The lifecycle below is what the user sees. Execution lives in a separate entity.

An application takes nine states to confirmation, with five ways out



Any state can also move to Cancelled or Expired. Terminal states are never overwritten.

application lifecycle · 9 states, 5 exits

The highlighted states match the three human gates. A site that cannot be automated leaves at Draft-only. A role the user skips leaves at Skipped. A user who sends outside the Guard becomes User reported sent. A crash or timeout after the submit intent becomes Submission uncertain and is never retried.

Runs (new). An `application_run` is one execution attempt, with the states queued, running, waiting_user, completed, failed and cancelled. A partial unique index allows one active run per application. Each run holds a lease with a heartbeat and an ordered step log. A missing fact, a CAPTCHA or an MFA prompt moves the run to waiting_user. Resuming creates a new run that starts from a fresh read of the page, never from remembered browser state.

Invariants.

1. Only the workflow engine writes lifecycle state, and every transition records actor and reason.
2. Cancellation is checked before every consequential action. The Guard checks it again at the click.
3. A new run reconciles with the live page before it acts.
4. An approval is consumed once and never reused.
5. A terminal state is never overwritten. A correction is a new event.

Data model additions

These changes sit on top of v1 section 23. Every table carries `user_id` and row-level security.

Entity	Change	Key fields
`profile_facts`	Becomes immutable and append-only	`claim_scope`, `supersedes_fact_id`, `source_type` including `user_interview`
`profile_versions`	New. A snapshot is a set of fact ids.	`fact_ids`, `created_at`
`watchlist_companies`	New. Drives discovery.	`platform`, `board_name`, `careers_url`, `last_synced_at`, `etag`

Entity	Change	Key fields
`jobs`	Gains platform identity and pay	`platform`, `board_name`, `posting_id`, compensation range, `closed_at` set when a posting disappears from its feed
`applications`	Lifecycle only. Execution moves out.	`state`, `fit_label`, `week_budget_slot`, `profile_version_id`, `resume_variant_id`
`resume_variants`	New	`base_document_id`, `diff`, `document_sha256`, `approved_at`
`application_runs`	New. One execution attempt.	`state`, `runner_device_id`, `lease_expires_at`, `heartbeat_at`
`run_steps`	New. Ordered step log without secret values.	`seq`, `action`, `field_key`, `value_hash`, `value_len`, `verification`
`submit_intents`	New. Written before the click.	`application_id`, `fence`, `payload_hash`, `runner_device_id`
`approval_records`	Rebound to the canonical payload	`payload_hash`, `runner_device_id`, `expires_at`, `consumed_at`, `revoked_at`
`outcome_events`	New. Append-only.	`stage`, `source` (adapter, user, email), `evidence_id`, `occurred_at`
`contacts`	New. Entered by the user, never scraped.	`name`, `company`, `relationship`
`follow_ups`	New	`application_id`, `due_at`, `draft`, `status`
`runner_devices`	New	`name`, `paired_at`, `last_seen_at`, `revoked_at`

Rules enforced by the database, not by code review.

- A partial unique index allows one active run per application.
- `submit_intents` is unique on application id and fence.

- Consuming an approval and recording the submit intent happen in one transaction, and the approval must be unexpired and unconsumed.
- `outcome_events` and `profile_facts` reject updates and deletes. Account deletion runs through a documented erasure path instead.

Notifications and assistance

Channel order. In-app inbox first, then email, then web push, then WhatsApp in a later release. The dashboard stays the source of truth, and every notification link requires a signed-in session, as in v1.

Assistance gets simpler. Because the runner is local, a CAPTCHA or MFA prompt is just a paused, visible browser window. The runner shows a banner that says "Complete this step, then press Resume". There is no remote-browser streaming and no ten-minute worker lifecycle. If the user closes the window, the run ends and the next run re-reads the page. Task types from v1 stay, with `fact_interview` added. The answer scope still defaults to this application only.

WhatsApp in a later release, with its real rules. WhatsApp suits users who live in it, so it earns a place later. It also has hard constraints:

- Free-form messages work only within 24 hours of the user's last message. Outside that window, only templates approved in advance by Meta can be sent, and approval can take up to a day.
- Keep every template strictly utility: "Your application to Example Co needs your input. Open Lintel to continue." One promotional line turns a template into marketing.
- Since 1 October 2026, in-window free-form replies and utility templates sent in response to a customer are billable. Send one daily digest by default and an immediate message only when a run is blocked.
- v1's message rules carry over: link only, no résumé content, no OTPs, no sensitive answers. A reply of "yes" can acknowledge a reminder but can never approve an application.
- Opt-in, verified phone association, easy opt-out, webhook signature checks, event deduplication and suppression after resolution all stay as written in v1 section 19.

Lintel is a tool that serves one user's own job search, not a general-purpose AI assistant offered through WhatsApp, which is the category Meta's policy restricts. Confirm this against Meta's current terms before launch.

Security, privacy and retention

The local-first design removes the largest risk in v1. Playwright's own documentation warns that saved browser state may contain cookies and headers that could be used to impersonate the user. In Lintel that state never leaves the user's machine, so the cloud has no logins to protect, leak or delete.

What the cloud holds. Facts, drafts, documents with hashes, approvals, run step logs without secret values, outcome events, and redacted evidence. It holds no cookies, no passwords and no browser profile.

Controls added or changed.

- **Device pairing.** The runner pairs with a device code and receives a per-device token stored in the operating system keychain. The user can revoke any device from the dashboard.
- **Signed approvals.** The server signs the payload hash, device id and expiry. The Guard verifies the signature before it lets a click through, so a stale or tampered payload cannot pass.
- **Dedicated browser profile.** The runner uses its own profile, never the user's everyday one. The user signs in to destination sites there, if they choose to, and the runner never reads other sites' cookies.
- **Egress allowlist.** The runner navigates only to hosts in the registry plus the asset hosts a reviewed page needs. It uploads only documents on the approved list, matched by hash.
- **Evidence by default is text, not pictures.** Screenshots are off. On a confirmation page the runner stores the matched text, the reference number and a page hash. The user can attach a screenshot manually.
- **Prompt-injection and untrusted content** rules from the harness section apply to all job text and page text.

Retention changes against v1 section 26.

Data	v1	v2
Browser authentication state	Encrypted in cloud, deleted at session end	Never in the cloud. It stays in the local profile under the user's control.

Data	v1	v2
Diagnostic screenshots	Redacted, seven days	Off by default. User-attached evidence stays with the application until deleted.
Run step logs	Operational logs, 14 days	Kept 90 days with the application drafts, and contain no secret values
Everything else	As written	Unchanged

Privacy posture. Lintel handles sensitive personal data, so the privacy notice must match the real list of processors: the model provider, email, storage and, later, the messaging provider. Export and deletion stay in the product from the first release. Deleting an account hard-deletes facts, drafts, documents and step logs. Backups age out on the documented schedule, and deletion cannot recall applications already sent to employers, which the product says plainly. Treat GDPR and India's data-protection law as design targets and get a legal review before any public beta.

Quality gates and metrics

The headline outcome is interviews per sent application, reported per user with its uncertainty. It is a measure to watch, not a target to promise, because a single user's sample is small and Lintel cannot prove it caused any interview. The metrics Lintel can hold to a hard line are about trust.

Metric	Definition	Target (proposed)
Unsupported claims in sent applications	Sentences in an approved payload that fail the token or scope check, or that the user flags	0
Unapproved submissions	Sends with no matching approval record	0
Duplicate sends	Two sends of one application on Lintel's authority	0

Metric	Definition	Target (proposed)
Audit coverage	Sent applications with an approval, a submit intent and an outcome event	100%
User time per application	Minutes from the fit decision to approval, after the first application	Under 10 at the median, measured against a manual baseline
Draft correction rate	Share of drafted answers the user materially edits	Measure first, set a target after the pilot
Fit-brief acceptance	Share of apply recommendations the user accepts	Measure first
Uncertain outcome rate	Sends ending without clear confirmation	Measure first
Interviews per sent application	From the outcome loop	Report only, with a confidence interval

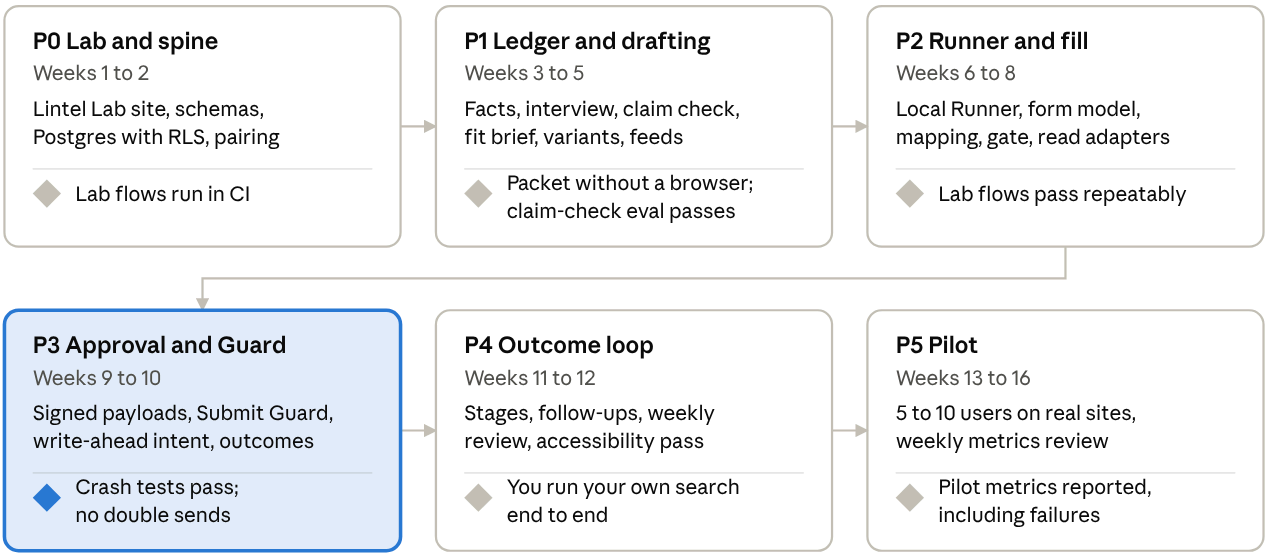
Test layers.

1. **Lintel Lab.** A local test site with every awkward case from v1 section 29.2, plus form skins that copy the patterns of Greenhouse, Lever and Ashby: conditional fields, multi-step flow, file upload, validation errors, session expiry, a CAPTCHA placeholder, an unexpected modal, and a timeout right after submit. It is the repeatable ground truth for agent work and a showpiece in its own right.
2. **Property tests.** Changing any single field of a payload changes its hash. Random event sequences never produce two sends. A stale fencing token is always rejected.
3. **Crash tests.** Kill the runner at each step of the submit sequence. Assert that the result is uncertain and that nothing is resent.
4. **Adversarial suite.** v1's list plus job text and page text that try to change tools, reveal data or trigger uploads. Your ROPE benchmark is a natural source of injection cases.
5. **Claim-check evaluation.** Synthetic profiles and jobs built to tempt fabrication: project versus employment, overlapping dates, "familiar with" versus "expert", missing sponsorship answers. Score unsupported-claim leakage into sent text, and report the verifier's recall separately. If Vernier is ready, running this set through it and publishing the report is a strong demonstration of both products. If not, a small script is enough.

Beta exit criteria (carried from v1, proposed). At least 90% preparation completion and 95% field accuracy on Lintel Lab flows, zero unapproved sends, zero cross-user exposure, zero automatic retries of uncertain sends. Live-site targets come from pilot measurements, not from fixtures.

Delivery plan

Six phases, each ending in a gate you can test



Diamond marks the exit gate of each phase. Estimates are proposed for one full-time developer.
Later: WhatsApp, email-based outcome detection, hosted runner, more adapters.

delivery roadmap · 6 phases, 6 gates, about 16 weeks

v1 estimated eight to twelve weeks for one experienced developer to reach a controlled beta. v2 adds the Local Runner, the Submit Guard, résumé variants, the Fact Interview and the outcome loop, and removes hosted browsers and early WhatsApp work. The proposed estimate is about 16 weeks to a pilot for one full-time developer. It is an estimate, not a validated figure, and the order matters more than the dates: the Lab first, trust mechanisms before breadth, and live users last. The accent phase is the one the product stands on, so it is the one not to compress.

Mapping to v1's releases. Release A is P1 and P4. Release B is P2 and P3. Discovery from v1's release C moves into P1 as the company watchlist, and WhatsApp becomes a later release D.

Showcase package

Most portfolio projects show that a model can fill a form. Lintel can show something rarer: a system that is designed to be wrong safely, with proof. Four things make it a strong project to put in front of a hiring manager.

1. **A trust mechanism with evidence.** Property tests, crash tests and a claim-check evaluation, with numbers you publish.
2. **A real loop with real users.** A pilot and outcome data, not a demo video.
3. **Honest platform engineering.** The product is shaped by what Greenhouse, Lever and Ashby actually allow, and says so.
4. **Systems depth.** Fencing tokens, write-ahead intents, row-level security, device pairing and signed approvals, all in one coherent product.

Repository layout. One monorepo, no more parts than the architecture needs.

```
lintel/
  apps/web         Next.js dashboard
  apps/api         FastAPI service
  apps/runner      Local Runner with Playwright
  packages/schemas  Pydantic models, generated TypeScript types
  packages/trust   claim check, payload hashing, approval signing
  lab/             Lintel Lab test site
  evals/           claim-check eval set, injection cases
  docs/            PRD, architecture, decisions, Trust Report
```

The three-minute demo.

1. Paste a careers URL. Lintel detects the platform and adds the company to the watchlist.
2. Open a fit brief with one gap. Answer the Fact Interview question and watch the draft cite the new fact.
3. Edit a draft sentence to claim something unconfirmed, such as leading a team. The claim check blocks it and says why.
4. Fill a Lintel Lab form. Approve. Change one answer in the browser and click Submit. The Guard blocks the click and lists the difference.
5. Kill the runner mid-submit. The application shows Submission uncertain and nothing is resent.
6. Open the Trust Report with the measured numbers and the list of what Lintel will not do.

The Trust Report. A public page with the claim-check results, the crash-test matrix, the Lab pass rates and an honest limits section. Publishing failures next to successes is what makes the numbers believable.

Resume bullets. Write these only after you have measured the numbers, and keep the claims as small as the evidence. Templates, with blanks to fill:

- Built Lintel, a supervised job-application agent with a confirmed-fact ledger, per-sentence claim checks and approval-bound submission using a write-ahead intent. [N] crash scenarios pass with zero double sends.
- Designed a local-first Playwright runner with a Submit Guard that compares live form values to a signed payload hash before the user's click. It blocked [N] of [N] injected mismatches in the Lab suite.
- Ran a pilot with [N] users. Median time per application fell to [N] minutes after the first, with [N] unsupported claims in sent text.

Place Lintel high in the projects section, in two lines at most, and list on the primary skills line only the tools you used hands-on in this build.

What to open-source. The Lab, the schemas, the trust package and the evals. Keep the hosted app in private beta. Two posts carry the story: why Lintel never clicks Submit for you, and a claim check for job applications.

Open decisions

Six calls are yours. Each has a recommendation, and none blocks starting the Lab.

Decision	Recommendation	What it changes
Name	Lintel, after a domain, app-store and trademark check. Fairhand is the fallback.	Branding, repo and CLI names
Execution model	Local Runner by default, hosted runner later	Removes cloud login custody and remote-browser work. Accepts that users install a small app.

Decision	Recommendation	What it changes
First live destinations	Build Lab skins for Greenhouse, Lever and Ashby. Enable real destinations one at a time, starting with the platform most of your own target companies use.	Order of adapter work and pilot realism
Beta scope	Private pilot of 5 to 10 users first. Legal review before any public beta.	Timeline, privacy notice, support load
Résumé variants	Reorder and reword existing bullets only, shown as a diff the user approves. No new claims, no new sections.	The trust story, and the claim-check surface
Model provider	Claude through the Anthropic API behind the provider interface, as in your other projects	Cost model and the eval baseline

Things this document could not verify. Whether each destination's terms allow assisted filling, which has to be reviewed per destination before it is enabled. How often real Greenhouse tenants show CAPTCHA, which comes from a competitor's test report and needs your own measurement. And the numbers marked proposed: approval lifetime, weekly budget, run limits, schedule.

Sources

Pages read for this document as of 21 October 2026, except the Greenhouse, AlternativeTo and PeerPush entries, which come from search results only. Vendor pages that compete in this market are marked, because they have an interest.

- Jobscan, auto-apply tools in 2026: callback ranges by tool category, platform detection, tailoring as the main factor. Jobscan sells a competing product.
- FastApply, bots tested in 2026: CAPTCHA walls on some Greenhouse tenants, failures on Workday. The author runs a competing product.
- Resumly, auto-apply tools ranked: user reports of invented skills in a competitor's output, and the current competitor field. Resumly sells a competing product.
- Greenhouse support, API overview: submit endpoint requires authentication.

- 100Hires, Lever API guide: public postings read, keyed apply endpoint, limits and gaps. Vendor page, claims attributed to Lever's own documentation.
- JobsPipe, Ashby API: public job-board endpoint, compensation flag, no customer discovery. Vendor page citing Ashby's developer documentation.
- Playwright, authentication: warning on saved browser state and what it covers.
- Courier, WhatsApp Business messaging: 24-hour window, templates, categories, billing change from 1 October 2026. Provider-side guide, to be checked against Meta's current documentation.
- AlternativeTo, Countersign and PeerPush, Foothold: name conflicts found in search results. Name checks were quick searches, not a trademark clearance.

The ApplyRelay v1 PRD is the base document throughout.